Block 4 – Session 1

Data Centers & Network Virtualization

Arquitectura de Xarxes

Universitat Pompeu Fabra

Outline

What Is the Cloud and How Is It Formed?

Every time you use Netflix, Gmail, or ChatGPT,
your request travels to a building full of servers.
What happens inside that building?

Learning objectives:

1. Understand why data centers exist and what they contain
2. Explain virtualization and distinguish VMs, hypervisors, and containers
3. Describe how networks are virtualized (vNICs, vSwitches, overlays)

Block 4 Roadmap

	Session 1 (today)	**Session 2**
Theme	*The infrastructure*	*The business model*
Topics	Data centers VMs & hypervisors VMs vs containers (overview) vNICs, vSwitches, vRouters Multi-tenancy & VLANs Overlay networks (VXLAN)	From virtualization to cloud NIST cloud definition IaaS / PaaS / SaaS VPC architecture Security groups & ACLs Governance & GDPR

Outline

What Is a Data Center?

- A **data center** is a facility housing computer systems and networking equipment
- Purpose: run applications and store data **reliably, at scale, 24/7**
- Ranges from small server rooms to **warehouse-scale** buildings

Who operates them?

- **Enterprises**: banks, hospitals, universities (their own IT)
- **Cloud providers**: AWS, Azure, GCP (sell capacity to others)
- **Colocation**: you rent space; provider handles power and cooling

Source: Barroso, Clidaras & Hölzle, *The Datacenter as a Computer*, 3rd ed., Morgan & Claypool, 2019.

A Data Center from the Inside

Rows of server racks, each containing dozens of servers, connected by structured cabling. Source: Cisco, *What Is a Data Center?*, cisco.com.

Inside a Data Center: Five Pillars

- **Compute**: CPUs that execute workloads (web servers, databases, analytics)
- **Accelerators**: GPUs and TPUs for AI/ML training and inference
  - The explosive growth of AI has made GPUs a critical data center resource
- **Storage**: SSDs and HDDs holding persistent data
- **Network**: switches, routers, and structured cabling connecting everything
- **Facilities**: redundant power supplies, cooling systems, physical security

The Problem with Physical Infrastructure

- Traditional model: **one server = one application**
- Typical server utilization: only **10–15%** of capacity
- Adding capacity means buying, racking, cabling $\rightarrow$ **weeks or months**
- Hardware failure takes down the entire service
- No way to scale down when demand drops

Key insight: most servers sit idle most of the time. We are paying for hardware we barely use.

From Waste to Efficiency: Consolidation

- **Consolidation** = run multiple workloads on fewer physical machines
- Benefits:
  - Higher utilization: **60–80%** instead of 10–15%
  - Lower energy, cooling, and physical space costs
  - Simpler infrastructure management
- Trade-off: shared hardware requires **strong isolation** between workloads
- The enabling technology? **Virtualization**

Analogy: instead of one person per house, we build apartment buildings. Shared structure, private spaces.

Scalability and Elasticity

- **Scalability**: ability to grow resources as demand increases
- **Elasticity**: ability to grow *and shrink* automatically
- Physical infrastructure provides **neither** in practice:
  - Cannot add servers in minutes
  - Cannot return servers when demand drops
- Virtualization enables both $\rightarrow$ foundation of **cloud computing** (Session 2)

Discussion: Data Centers

A company runs 50 servers, each at 10 percent utilization.
That is 50 machines doing the work of 5.
What would you do to reduce waste?

Hints: consolidation, fewer physical machines, higher utilization, isolation between workloads.

Outline

What Is Virtualization?

- **Virtualization** makes it possible for a single computer to act like many
- A software layer called **hypervisor** divides the physical resources (CPU, RAM, disk, NIC) into isolated **virtual machines**
- Each VM gets its own share and believes it owns dedicated hardware

Virtualization began in the 1960s as a technology for time-sharing on mainframe computers. It gained popularity in the 2000s as organizations looked for ways to make the most of their computing resources.

Source: Red Hat, *What is a Virtual Machine?*, redhat.com.

Virtual Machines (VMs)

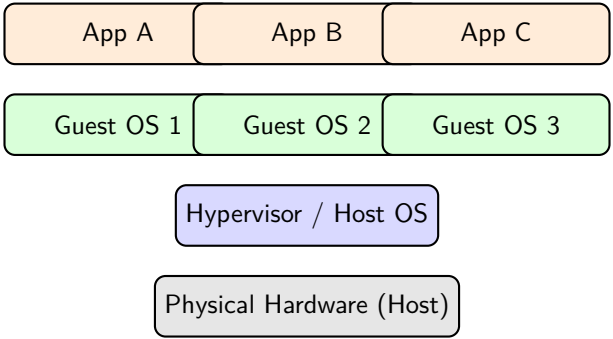
- A VM is an **isolated computing environment** with its own CPU, memory, network interface, and storage
- Runs a **full operating system** (Linux, Windows, etc.)
- Completely **isolated** from other VMs on the same host

Key properties:

- **Encapsulation**: entire VM state stored as files $\rightarrow$ easy to copy, move, backup
- **Hardware independence**: the VM does not care about the underlying hardware
- **Snapshotting**: save and restore VM state at any point in time

Source: Popek & Goldberg, "Formal Requirements for Virtualizable Third Generation Architectures," *CACM*, 1974.

Host and Guest Systems



- **Host**: physical machine providing CPU, RAM, disk, NIC
- **Guest**: virtual machine running its own OS on the host
- Each guest believes it has **dedicated hardware** (abstraction)

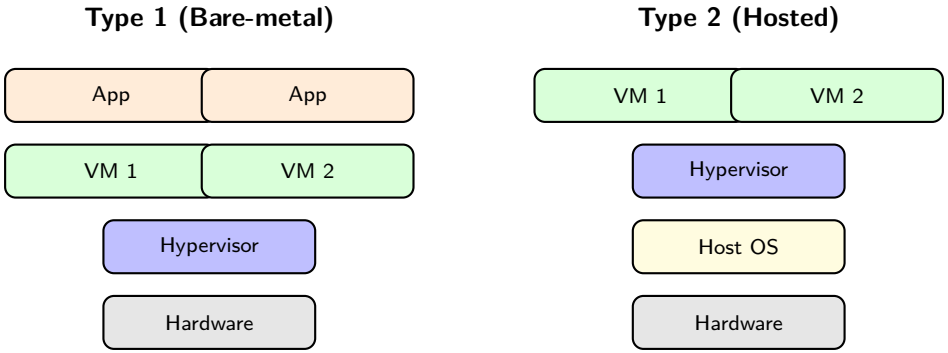
Hypervisors: The Key Enabler

- A **hypervisor** (or VMM) manages and allocates physical resources to VMs
- Two types:

	Type 1 (Bare-metal)	**Type 2 (Hosted)**
Runs on	Directly on hardware	On top of a host OS
Performance	Near-native	Some overhead
Use case	Data centers, production	Development, testing
Examples	VMware ESXi, KVM, Hyper-V	VirtualBox, VMware Workstation

Source: Barham et al., "Xen and the Art of Virtualization," *SOSP*, 2003.

Type 1 vs Type 2: Visual Comparison



- Type 1: hypervisor **replaces** the OS → less overhead, used in production
- Type 2: hypervisor runs **as an application** → easier to set up, used for dev/testing

VMs vs Containers: Overview

Virtual Machines

- Include a **full guest OS** per instance
- Size: **GBs** (full OS image)
- Boot time: **minutes**
- Strong isolation (hardware level)
- Can run **different operating systems**

Containers

- **No guest OS**; share the host kernel
- Size: **MBs** (app + dependencies only)
- Boot time: **seconds**
- Good isolation (kernel namespaces/cgroups)
- All containers share the **same OS kernel**

This is just an overview. We will do a deep dive into container internals and networking in **Block 5**.

Source: Red Hat, *Containers vs VMs*, redhat.com.

When to Use VMs vs Containers

- **Use VMs when:**
  - You need **different operating systems** (Linux + Windows on the same host)
  - Strong **security isolation** is critical (e.g., different tenants)
  - Legacy applications that require a specific OS environment
- **Use containers when:**
  - Running **many instances** of similar applications
  - **Fast startup** and scaling matters
  - **Microservices** architecture (each service in its own container)
- **In practice**: most organizations use **both** together
  - Containers run inside VMs for defense in depth

Microservices and container networking: **Block 5**.

Discussion: Virtualization

You need to deploy 200 instances of the same web application that must scale up and down every few minutes. Would you use VMs, containers, or both? Why?

Hints: boot time, resource overhead, isolation needs, how fast you need to scale.

Outline

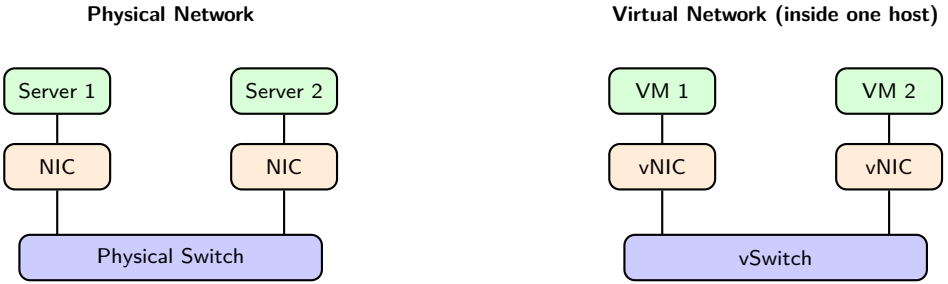
From Physical to Virtual Networks

- You already know how physical networks work (Blocks 1 to 3):
  - NICs, switches, routers, VLANs, routing protocols
- When we virtualize servers, we also need to **virtualize the network**
- The same concepts apply, but implemented **in software** instead of hardware

Physical	**Virtual equivalent**
NIC (network interface card)	**vNIC** (virtual NIC)
Switch	**vSwitch** (virtual switch)
Router	**vRouter** (virtual router)
Cable	Internal software path

Key idea: if VMs think they have real hardware, they also need a **real-looking network**.

Physical vs Virtual: Side by Side



- Left: two physical servers connected by a physical switch and cables
- Right: two VMs connected by a virtual switch, all inside **one physical host**

Virtual Network Interfaces (vNICs)

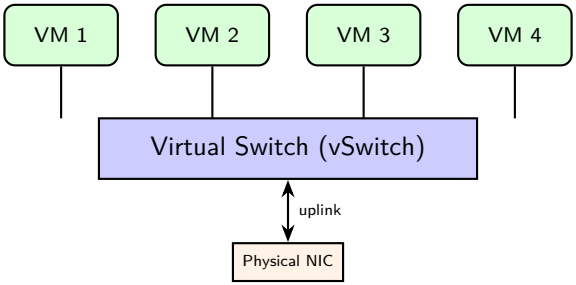
- A **vNIC** is a software-emulated network interface card
- From the guest's perspective: behaves exactly like a real NIC
- Each VM gets **one or more vNICs**

Key characteristics:

- Has its own **MAC address** (software-assigned by the hypervisor)
- Has its own **IP address** configuration
- Connected to a **virtual switch** on the host
- Multiple vNICs per VM enable **multi-network** connectivity

Same MAC/IP concepts from Blocks 1 to 3, now virtualized inside the hypervisor.

Virtual Switching



- Connects VMs on the same host, works like a **physical L2 switch**
- Maintains **MAC address table**, forwards frames by destination MAC
- Connected to physical NIC via **uplink** for external traffic

Types of Virtual Switches

- **Standard vSwitch**: basic, built into the hypervisor
- **Distributed vSwitch**: spans multiple hosts, centralized management
- **Open vSwitch (OVS)**: open source, programmable, supports **SDN**

- Supports **VLANs** for traffic segmentation between VMs
  - Same 802.1Q from Block 3, now inside the hypervisor

OVS and SDN: we will explore how switches become programmable in **Block 5**.

Source: Pfaff et al., "The Design and Implementation of Open vSwitch," *NSDI*, 2015.

Bridging Modes

Three ways to connect a VM to the outside world:

Mode	VM visibility	External access?	Use case
Bridged	Same subnet as host	Yes (direct)	Production servers
NAT	Hidden behind host IP	Outbound only	Dev/testing
Host-only	Only sees the host	No	Isolated labs

- **Bridged**: VM appears directly on the physical network
- **NAT**: VM traffic translated through the host's IP (same NAT from Block 3)
- **Host-only**: completely isolated from external networks

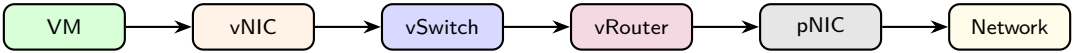
Virtual Routing and NAT

- A **virtual router** forwards traffic between virtual subnets
- Same function as a physical router, implemented **in software**
- Handles:
  - Routing between VMs on **different subnets**
  - **NAT** for VMs accessing external networks
  - Acting as **default gateway** for VMs

Same NAT concept from Block 3, now managed by the hypervisor instead of a physical device.

In Session 2 we will see how cloud providers offer NAT as a **managed service** (NAT Gateway).

Packet Flow: Virtual to Physical



1. VM generates packet with **virtual MAC/IP** as source
2. vNIC passes frame to **vSwitch** (L2 forwarding)
3. If destination is another subnet $\rightarrow$ **vRouter** (L3 forwarding)
4. vRouter may apply **NAT** (replace private IP with host IP)
5. Frame exits through **physical NIC** to the external network

Discussion: Virtual Networks

Two VMs on the same host want to communicate.
Does their traffic ever leave the physical machine?
What if they are on different subnets?

Hints: vSwitch handles same-subnet traffic locally; vRouter needed for different subnets; traffic may still stay inside the host.

Outline

Why Isolation Matters

- Data centers host workloads from **multiple tenants** (users, departments, companies)
- **Multi-tenancy**: shared infrastructure, but each tenant expects:
  - **Performance isolation**: my traffic is not affected by yours
  - **Security isolation**: you cannot see or access my data
  - **Address independence**: we can both use `10.0.0.0/24`

Without isolation: broadcast storms, ARP spoofing, IP conflicts, resource starvation.

VLANs in Virtual Environments

- Same **802.1Q** concept from Block 3, now applied to virtual switches
- Hypervisors assign VMs to specific VLANs via **vSwitch port groups**
- Traffic tagged with VLAN IDs: tenants separated at L2

Limitation: VLAN ID is 12 bits $\rightarrow$ max **4,096 VLANs**

Problem: 4,096 VLANs is **not enough** for a large cloud with thousands of tenants.

Source: IEEE 802.1Q-2022, "Bridges and Bridged Networks."

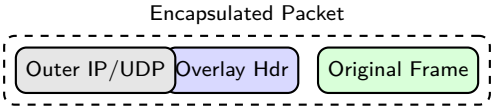
Micro-Segmentation

- Beyond VLANs: **per-VM or per-workload firewall policies**
- Goal: **least privilege**, each VM should only reach what it strictly needs
- Key concept in modern **zero trust** security architectures

In Session 2 we will see how cloud providers implement this with **Security Groups** (per-instance) and **Network ACLs** (per-subnet).

Overlay Networks: Concept

- An **overlay network** is a virtual network built **on top of** the physical one
- Uses **encapsulation**: wrap virtual frames inside physical packets



- The physical network only sees the **outer headers**
- The virtual (tenant) traffic is hidden inside $\rightarrow$ **full isolation**

VXLAN: Overview

- **VXLAN** (Virtual eXtensible LAN): most widely used overlay protocol
- Encapsulates L2 frames in **UDP packets** over the physical network
- Uses a **24-bit VNI** (VXLAN Network Identifier)
  - $2^{24} = 16$ **million** virtual networks (vs 4,096 VLANs!)
- Endpoints: **VTEPs** (VXLAN Tunnel Endpoints)
  - Encapsulate at source, decapsulate at destination

Source: IETF RFC 7348, "Virtual eXtensible Local Area Network (VXLAN)," 2014.

VXLAN: How It Works

- VM A sends a frame → VTEP 1 wraps it in UDP with a VNI
- Travels over the physical network as a regular UDP packet
- VTEP 2 strips outer headers, delivers original frame to VM B
- VMs on the **same VNI** form an isolated L2 domain

Overlay Benefits for Scalability

- **16 million virtual networks** (vs 4,096 VLANs)
- Physical network does not need to know about tenants
- VMs can **migrate** across hosts without reconfiguring the underlay
- Decouples **virtual topology** from physical topology

This is the foundation of cloud networking. In **Session 2** we will see how providers build **VPCs** (Virtual Private Clouds) on top of overlays.

Source: IETF RFC 8926, "Geneve: Generic Network Virtualization Encapsulation," 2020.

Outline

Key Takeaways

1. **Data centers** house compute, accelerators (GPUs), storage, network, and facilities
2. Physical infrastructure is **rigid, underutilized, and slow to scale**
3. **Hypervisors** (Type 1 and 2) enable multiple VMs on one host
4. **Containers** are lighter than VMs but share the kernel (deep dive in Block 5)
5. Virtual networks mirror physical ones: **vNICs, vSwitches, vRouters**
6. **Multi-tenancy** requires isolation $\rightarrow$ VLANs, overlays, micro-segmentation
7. **VXLAN** extends VLANs from 4K to 16M networks using encapsulation

Next Session: Preview

In **Session 2** we build on today's foundation:

- Virtualization is the technology; cloud is the **business model**
- NIST definition and **5 essential characteristics**
- Service models: **IaaS / PaaS / SaaS**
- **VPC** architecture: subnets, route tables, gateways
- Cloud security: **Security Groups + Network ACLs**
- Governance, **data sovereignty**, GDPR

Discussion

You manage a data center with 5,000 tenants.
Each tenant wants their own isolated network.
Why can't you use VLANs alone?
What would you use instead, and why?

Hints: VLAN ID limit, overlapping IP ranges, VM mobility.

References

1. Popek & Goldberg, "Formal Requirements for Virtualizable Third Generation Architectures," *CACM*, vol. 17, no. 7, 1974.
2. Barham et al., "Xen and the Art of Virtualization," *SOSP*, 2003.
3. VMware, "Understanding Full Virtualization, Paravirtualization, and Hardware Assist," 2007.
4. Pfaff et al., "The Design and Implementation of Open vSwitch," *NSDI*, 2015.
5. IEEE 802.1Q-2022, "Bridges and Bridged Networks."
6. IETF RFC 7348, "Virtual eXtensible Local Area Network (VXLAN)," 2014.
7. IETF RFC 8926, "Geneve: Generic Network Virtualization Encapsulation," 2020.
8. Barroso, Clidaras & Hölzle, *The Datacenter as a Computer*, 3rd ed., Morgan & Claypool, 2019.
9. NIST SP 800-125, "Guide to Security for Full Virtualization Technologies," 2011.
10. Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.
11. Red Hat, "What is a Virtual Machine?", redhat.com/en/topics/virtualization/what-is-a-virtual-machine.
12. Red Hat, "Containers vs VMs," redhat.com/en/topics/containers/containers-vs-vms.
13. Cisco, "What Is a Data Center?", cisco.com/site/us/en/learn/topics/computing/what-is-a-data-center.html.